Lecture 28.14: Decision Tree Regressor in Scikit-Learn

📌 Topic Overview

In this final submodule, we implement `DecisionTreeRegressor` in Python using Scikit-Learn on the **Diabetes dataset** (`load_diabetes(as_frame=True)`), observe severe overfitting in continuous space (negative R^2 on test data!), visualize the tree with `plot_tree`, and resolve it using pre-pruning.

🩺 Dataset & Setup

- **Dataset**: `load_diabetes(as_frame=True).frame` (442 samples, 10 physiological features).
- **Target**: Continuous measure of disease progression one year after baseline.
- **Split**: `train_test_split(X, y, test_size=0.3, random_state=42)`.

⚠️ Unpruned Regression Tree: Extreme Overfitting

When we fit an unrestricted `DecisionTreeRegressor()`:

- **Train MSE**: `0.0`
- **Train R^2**: `1.0` (100% memorization!)
- **Test MSE**: `5530.51`
- **Test R^2**: `-0.024` (Negative R^2!)

Why is Test R^2 Negative?

A negative R^2 means the unpruned tree is performing **worse than a simple horizontal line predicting the overall mean** ($\bar{y}$)! The tree created hundreds of tiny leaves to fit individual training outliers, which fail disastrously on test points.

✂️ Fixing the Model with Pre-Pruning

By constraining the tree using `max_depth` and `min_samples_leaf`:

- The tree stops memorizing noise.
- The test R^2 turns from negative to positive.

- The test MSE drops significantly.

💻 Python Implementation from Lecture

```
from sklearn.datasets import load_diabetes
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error, r2_score

def evaluate_diabetes_regression_trees():
    """
    Trains and compares unconstrained vs regularized DecisionTreeRegressor
    on the Scikit-Learn Diabetes dataset.
    """
    # 1. Load data and split
    X, y = load_diabetes(return_X_y=True)
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )

    # 2. Unpruned Regressor (Overfitting)
    unpruned = DecisionTreeRegressor(random_state=42)
    unpruned.fit(X_train, y_train)

    # 3. Regularized Regressor (max_depth=3, min_samples_leaf=15)
    regularized = DecisionTreeRegressor(max_depth=3, min_samples_leaf=15, random_s
    regularized.fit(X_train, y_train)

    # 4. Compute metrics
    metrics = {
        'unpruned_train_r2': r2_score(y_train, unpruned.predict(X_train)),
        'unpruned_test_r2':  r2_score(y_test, unpruned.predict(X_test)),
        'unpruned_test_mse': mean_squared_error(y_test, unpruned.predict(X_test)),
        'reg_train_r2':      r2_score(y_train, regularized.predict(X_train)),
        'reg_test_r2':       r2_score(y_test, regularized.predict(X_test)),
        'reg_test_mse':      mean_squared_error(y_test, regularized.predict(X_test
    }
    return metrics

# --- Execute & Display Results ---
results = evaluate_diabetes_regression_trees()
print("=== Unpruned Regressor (Severe Overfit) ===")
print(f"Train R^2: {results['unpruned_train_r2']:.3f} | Test R^2: {results['unprun
print(f"Test MSE:  {results['unpruned_test_mse']:.2f}")

print("\n=== Regularized Regressor (max_depth=3, min_samples_leaf=15) ===")
```

```
print(f"Train R^2: {results['reg_train_r2']:.3f} | Test R^2: {results['reg_test_r2
print(f"Test MSE:  {results['reg_test_mse']:.2f}")
```

🔑 Key Takeaways

1. Unconstrained regression trees easily overfit, causing R^2 to become negative on test data.
2. `plot_tree` allows visual inspection of leaf sizes and split criteria.
3. Pre-pruning via `max_depth` and `min_samples_leaf` is essential to achieve positive generalization performance.